QR Code Generator

Design specification - svg-dom-graph

Revision 5 | 29 October 2026 | Implementation design, not code

A deterministic, explainable Model 2 QR encoder rendered as an interactive SVG graph. This revision resolves the six residual findings on Revision 4 without changing the encoding, snapshot, or UI architecture contracts.

Decision

Implement Version 1-L, byte mode, ASCII-only (0-17 bytes) in a pure Rust core. Show 59 immutable snapshots through an existing selection-toolbar anchor, a stable graph, and a demo-local three-state matrix viewport. Keep all new matrix and finite-field representations private to the demo.

Revision 5 change register

Review finding	Normative resolution
Mask mutation	flip_data() applies candidate XOR only to assigned Data/ErrorCorrection cells in a clone of the unmasked grid.
Redundant reservation bits	Remove reserved bitset; function reservation is derived from role and assignment state.
Scene diagram	Use a clear rectangular two-column layout diagram with explicit labelled relationships.
Snapshot storage	Vec; assert exactly 59 entries before committing a replacement.
First stage jump	Label index 0 button "Input"; it selects snapshot 0, not Unstarted.
External selection regression	Test set_selection followed by explicit apply_snapshot, including absence of an on_step callback.

Status: Approved design ready for Phase 0 after resolving Revision 4 residual items. Preserve unrelated local changes, particularly the uncommitted changelog.md.

1. Scope and invariants

1.1 Objectives

- Explain input encoding, padding, Reed-Solomon error correction, function modules, placement, masking and selection.
- Produce spec-conformant, scannable exports; make every intermediate state reproducible without SVG or browser dependencies.
- Support keyboard stepping, Prev, Restart, stage jumps, input replacement and assistive technology without rebuilding the scene on normal updates.
- Prototype the matrix locally; defer a public API until a second genuine client establishes common requirements.

1.2 Supported first release

QR Code Model 2, Version 1, level L: 21 x 21 modules; 19 data codewords (152 bits), 7 error-correction codewords (56 bits), 26 codewords total (208 bits). Byte mode indicator 0100 plus an 8-bit character count. ASCII U+0000..U+007F, no ECI; accept 0..17 bytes. UI may restrict ASCII controls for usability but the encoder must define them.

Quiet zone: Neither intermediate nor final interactive viewport includes a quiet zone. The export writer adds exactly four light modules along each edge, yielding 29 x 29 modules before pixel scaling. Never count quiet-zone cells in mask scoring.

1.3 Exclusions

Version selection, higher versions, numeric/alphanumeric/Kanji optimisation, non-ASCII/Unicode mapping, ECI, multi-block RS and interleaving, Micro QR, structured append, decoding and error correction on received symbols.

2. Prerequisite decisions

Component	Placement	Decision
BitGrid + MatrixView	Demo-local	Required: tri-state packed grid, separate module roles, bounded SVG paths.
Gf256	Encoder-private	Required for RS; byte arithmetic modulo 0x11D; not a public NodeValues variant.
BitWriter	Encoder-private	Required for MSB-first bits, padding and length checks.
QrModuleKind	Demo/core private	Required semantic role, separately optional until reserved or placed.
Codeword / score display	Existing node types	Reuse BitValues and integer nodes; no new scalar data type.

3. Matrix model: assignment versus reservation

Cell assignment and semantic role are distinct axes. A cell may be reserved for a format bit while its value is unknown. An untouched data slot has no role until codeword placement. Reservations protect cells from the data-placement traversal even when the colour is Unassigned.

```
enum CellState { Unassigned, Light, Dark }
enum QrModuleKind { Finder, Separator, Timing, DarkModule,
                    Format, Data, ErrorCorrection }
struct BitGrid {
    width: usize, height: usize,
    assigned: Vec<u64>, dark: Vec<u64>,
    roles: Vec<Option<QrModuleKind>>,
}
fn reserve(&mut self, row: usize, col: usize,
           role: QrModuleKind) -> Result<()>;
fn assign(&mut self, row: usize, col: usize,
          value: bool, role: QrModuleKind) -> Result<()>;
fn fill_reserved(&mut self, row: usize, col: usize,
                 value: bool, expected_role: QrModuleKind) -> Result<()>;
fn flip_data(&mut self, row: usize, col: usize) -> Result<()>;
fn get(&self, row: usize, col: usize) -> Option<CellState>;
```

The names/signatures above are design contracts, not claims of current repository API. Implementation may specialise fill_reserved for format fields. Seven u64 words per bitset cover 441 cells; two bitsets cost 112 bytes plus 441 optional roles. No separate reservation bitset exists. Keep padding bits zero.

3.1 Mutation rules and invariants

- reserve() stores role=Some(function role) without assigning a value. Only function roles (Finder, Separator, Timing, DarkModule, Format) may be reserved; reject conflicting roles or assigned cells. A function-reserved cell is identified by this role, not by a separate bitset.
- assign() applies only to a role=None, assigned=0 cell. It sets assigned=1, dark to the supplied Boolean and role=Data or ErrorCorrection; function roles must be written via fill_reserved().
- fill_reserved() requires an existing matching function role and assigned=0, then sets assigned and dark. At snapshot 23 call reserve() and then fill_reserved() for finder, separator, timing and fixed-dark colours; do NOT fill the format cells until a mask candidate is constructed.
- flip_data() requires assigned=1 and role=Data or ErrorCorrection, toggles dark, preserves assigned and role, and rejects all function/unassigned cells. Mask candidates clone the complete unmasked placed grid and apply this operation exactly once where the mask predicate holds.
- Data and ErrorCorrection roles are attached when their codewords are placed (snapshots 24..49); unplaced data slots have role=None and assigned=0. Role=None with assigned=1 is invalid. Format roles remain reserved-but-unassigned at 23 and are assigned separately in each candidate at 50..57.
- Invariant tests assert: dark implies assigned; each assigned cell has a role; function roles cannot be overwritten by assign() or flip_data(); Data/EC roles are always assigned; unassigned format cells retain their Format role. Unused high bits in the last word remain zero.

3.2 Renderer contract

Exactly three base paths contain assigned-dark, assigned-light and unassigned module geometry. A fourth consolidated role-outline path selects ONE semantic role at a time, controlled by the legend; a separate bounded overlay marks the active cell(s). Change fill/stroke and path geometry without creating a rectangle per cell. The legend has a None/All-off option; only a single role is ever emphasised at once. Accessible text names the selected role.

4. Encoder and symbol construction

4.1 Input, data codewords and error correction

Accept ASCII-only input, 0..17 bytes. Encode 0100, 8-bit count and bytes MSB-first; append up to four zero terminator bits, align to 8 bits, and alternate pad codewords EC / 11 (hex) to fill 19 bytes. Reject invalid inputs before committing a new UI state. Reed-Solomon uses GF(2^8), primitive polynomial 0x11D and degree-7 generator roots alpha^0..alpha^6. Divide 19 data codewords followed by 7 zeros to obtain 7 remainder bytes. Coefficient order is fixed and unit-tested.

4.2 Function modules and placement

Reserve finder/separator regions, timing patterns, both format copies and fixed dark module (row 13, col 8). Version 1 has no alignment pattern or version-information fields. Exactly 233 slots are function-reserved and exactly 208 remain for data/ECC. Starting with column pair (20,19) and row 20, traverse upward, then alternate downward/upward for each successive pair moving left; skip column 6. Place each of the 26 codewords MSB first.

4.3 Masks

For zero-based row r and column c, clone the unmasked placed matrix for each candidate, evaluate its predicate only at assigned Data/ErrorCorrection cells and call flip_data() on matches. Never alter function modules or mask a preceding candidate.

```
0: (r+c)%2 == 0            1: r%2 == 0
2: c%3 == 0                3: (r+c)%3 == 0
4: (r/2 + c/3)%2 == 0     5: (r*c)%2 + (r*c)%3 == 0
6: ((r*c)%2 + (r*c)%3)%2 == 0
7: ((r+c)%2 + (r*c)%3)%2 == 0
```

For every candidate, calculate format BCH using generator 0x537 and final XOR 0x5412, assign both format copies for THAT mask, then score. Choose the minimum penalty; ties select the lowest mask index. The final on-screen symbol remains exactly 21 x 21; export alone adds the quiet zone.

4.4 Format positions (row, column)

Format bit index i is the i-th least-significant bit of the 15-bit format word. Copy A: bits 0..5 at (0..5,8); bit 6 at (7,8); bit 7 at (8,8); bit 8 at (8,7); bits 9..14 at (8,5..0). Copy B: bits 0..7 at (8,20..13); bits 8..14 at (14..20,8). The fixed dark module (13,8) is not format information. These two placements are not mirrors.

5. Penalty scoring and independent oracle

5.1 Deterministic scoring

- N1: runs of at least five equal modules in rows/columns score 3 for the first five and +1 per additional equal module.
- N2: every 2 x 2 equal-colour square scores 3.
- N3: evaluate each row and column padded with FOUR virtual light modules on both ends; count all overlapping 11-cell windows matching 00001011101 or 10111010000; each scores 40. Out-of-bounds modules are light for N3 only.
- N4: 10 * floor(abs(2 * dark - total) * 10 / total) in integer arithmetic; total=441 for the on-screen symbol.

N3 edge treatment is normative for this demo, not an assertion that every reference encoder uses the same policy. Test patterns at both boundaries, interior matches, overlapping windows and negative cases.

5.2 Oracle fixtures and comparison contract

Generate offline reference fixtures using a pinned Python segno version, forced Model 2 Version 1, error correction L, byte mode, mask index 0..7, no automatic boosting, and the exact ASCII bytes. Capture data/EC codewords, both format copies, eight post-format pre-scoring candidate matrices, penalty breakdowns, chosen mask and final matrix. Measure segno N3 scoring under both border conventions before accepting the Phase 2 gate.

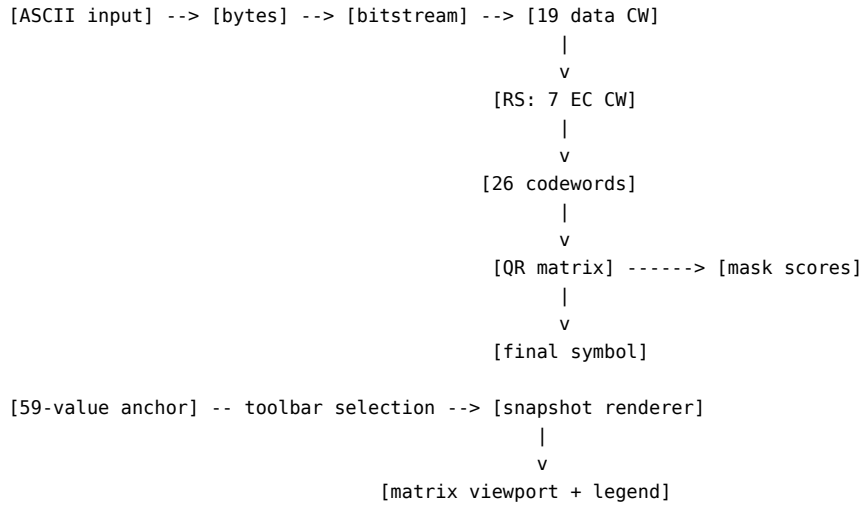
Acceptance: all eight identically indexed candidate matrices and format bits MUST match, independently of the selected mask. Compare final matrices only when both implementations choose the same mask. If they differ, document the scoring convention and each chosen mask/score, and verify each selected final matrix equals that implementation's corresponding checked candidate. No silent scoring changes to imitate the oracle.

5.3 Decoder verification

Independently decode exports using ZXing, and pyzbar where the runtime supports it. Test empty input, ordinary ASCII, punctuation and 17-byte capacity. Always include the export-only four-module quiet zone and integer-sized squares. Capture oracle version, configuration and fixture provenance in the test data.

6. Scene layout and DOM rendering

6.1 Fixed graph



A compact 59-cell integer anchor data node controls the step toolbar and is separate from the matrix. The matrix viewport occupies a reserved fixed-size right-hand region (below the graph at narrow widths), with a legend and score table. Each arrow is one graph edge, not a per-bit edge. The anchored region, score table and status strip do not move as descriptions grow; overflow panels scroll.

6.2 Bounded SVG representation

The 21 x 21 viewport has three consolidated filled paths: unassigned, light, dark. A single fourth outline path shows the currently selected role only; selecting a different role replaces its d attribute and optionally its stroke style. A bounded overlay group highlights the current cell range. Limit distinct DOM elements independently of module count; do not instantiate 441 rects or 441 graph nodes.

Between snapshots, regenerate base-path geometry when assignments change, emphasis geometry when the selected role or its cell set changes, and highlights only when highlights change. Snapshot rendering is whole-state and idempotent: do not depend on replay of earlier DOM deltas. A failed DOM write must surface an error and be retried from canonical snapshot on the next explicit step/refresh.

6.3 Accessibility

Provide labelled keyboard-operable toolbar, stage-jump buttons, role selector, mask score table and export. A concise aria-live announcement names stage and current action; avoid announcing 441 cells for each step. Detailed matrix rows/columns and module roles are opt-in. Never communicate roles solely through hue, and preserve scene focus on ordinary cell changes.

7. Snapshots, toolbar and stage jumps

7.1 Canonical state

```
enum ViewState { Unstarted, At(usize) }
struct DemoSnapshot {
    stage: Stage, label: String, matrix: BitGrid,
    highlights: Vec<Cell>, active_nodes: Vec<NodeId>,
    active_edges: Vec<EdgeId>, status: StageStatus,
}
// Exactly 59 immutable snapshots for each accepted input, stored as Vec<DemoSnapshot> with an asserted length of 59.
```

Indices	Count	Content
0-3	4	Input/validation; byte values; bitstream; 19 padded data codewords.
4-22	19	One Reed-Solomon division iteration per data codeword.
23	1	Function modules reserved; format cells reserved but unassigned.
24-49	26	Place one full codeword per snapshot (data or EC provenance attached).
50-57	8	One complete candidate per snapshot, including assigned mask-specific format bits before scoring; single viewport with score table.
58	1	Winning candidate, final 21 x 21 on-screen symbol, ready for export.

Unstarted is not snapshot 0. It shows an empty 21 x 21 baseline with no stale highlights or role assignments. At mask snapshot 50, previously reserved format cells become assigned for the first time. Each later mask snapshot derives from the same unmasked placed matrix and writes that mask's own format bits, never accumulates mask XOR operations.

7.2 Toolbar behaviour

Bind Scene::show_selection_toolbar to a fixed 59-value integer data node. Next from Unstarted goes to 0; Next from 58 is disabled; Prev from 0 and Restart from any active index go to Unstarted; Prev and Restart are disabled when Unstarted. Snapshot index equals the toolbar anchor's flat cell index. The callback renders the target snapshot after selection is committed; no second cursor exists.

7.3 Stage jumps via existing API

Stage-jump buttons labelled Input, Reed-Solomon, Function modules, Placement, Masks and Final target indices 0, 4, 23, 24, 50 and 58 respectively. Input selects snapshot 0 (not Unstarted); only Prev from 0 or Restart returns to Unstarted. For each button, use Scene::set_selection on the anchor with the appropriate natural 1-D cell selection. This API is already called by the repository's selection toolbar; externally changing selection updates toolbar button states. Then invoke the same demo snapshot-application function explicitly: external set_selection does not itself invoke the toolbar's on_step callback. Do not use fixed toolbar stride to jump between unequal-length stages, and do not add a Scene API.

8. Input replacement, lifecycle and failures

8.1 New valid input while stepping

Validate input first, construct a complete replacement Vec, assert len() == 59, then commit the new snapshot collection. Change the existing anchor node selection to Selection::None through Scene::set_selection and apply Unstarted once. Keep the anchor's 59 values and existing toolbar instance: do NOT rebuild the toolbar, graph or scene. Refresh visible input byte labels/status from the new input even while Unstarted, and clear previous matrix, scores, highlights and announced step.

If validation, computation or state construction fails, retain the last-good snapshots, anchor selection and visible state; show a non-destructive error. If the anchor-reset DOM operation fails, do not claim a successful input replacement; reconcile the displayed state from canonical committed data before accepting further steps. Test input changes at Unstarted, RS, placement, mask and final states.

8.2 Errors and teardown

Failure	Required handling
Unsupported input / length	Typed ASCII/capacity error; 17-byte maximum; leave last-good scene unchanged.
Conflicting reserve/assign	Typed grid invariant error; reject without partially committing grid state.
RS / placement count mismatch	Internal error with stage context; no successful symbol.
Partial SVG DOM mutation	Report failure; retry full canonical state on next explicit action.
Scene drop / teardown	Drop listeners; ignore queued actions naming obsolete SVG IDs.
Export failure	Report separately; keep interactive final symbol intact.

Treat user input as untrusted: update SVG text nodes rather than interpolating SVG markup. Bound input, matrix dimensions, snapshot count and path-string lengths. Preserve existing zoom/pan/focus for routine changes. A full rebuild, if ever necessary, follows established SVG-root swap conventions.

9. Tests and phase gates

9.1 Tests

- Encoder: input length 0/17/18, ASCII restrictions, headers, terminator, pad bytes, RS generator and seven-byte remainder fixtures.
- Grid: reserve before assignment, format cells reserved-unassigned at snapshot 23, no assignment into reserved cells through the ordinary data writer, and data-vs-EC provenance established only on placement.
- Symbol: exactly 233 function-reserved and 208 data/ECC locations, correct column-pair traversal, eight masks, both format copies, deterministic N1-N4 and tie-breaking.
- Oracle: compare codewords and all eight candidates with mask-specific format bits, conditional final-mask rule, documented N3 difference and decoder round trips.
- Navigation: 59 steps; index-0 Input jump differs from Restart; Next disabled at 58; Prev from 0; and input replacement at each stage.
- External selection regression: call Scene::set_selection on the anchor, assert toolbar state updates without an on_step invocation, then explicitly call apply_snapshot(index), and assert viewport, score/status, highlights and focus reflect that index; verify jumps 0/4/23/24/50/58.
- Rendering: bounded path count, unassigned visible, selectable single-role outline, highlight-only update, four-module export border, keyboard and screen-reader tests.

9.2 Delivery stages

Phase	Deliverable	Gate
0 - demo spike	Demo-local BitGrid, reserve()/assign()/fill_reserved()/flip_data(), four consolidated paths, legend, highlight overlay.	Prove tri-state and role reservation; inspect DOM mutations, focus, selected-role contrast.
1 - pure encoder	ASCII bit packing, 19 data bytes, Gf256 and 7 RS bytes.	Independent codeword fixtures; no browser dependency.
2 - QR symbol	233 reserved slots; placement; 8 masks; N1-N4; format bits.	All candidate matrices match oracle; record N3 policy.
3 - integration	Fixed graph; 59 snapshots; toolbar; stage jumps; input reset.	Deterministic navigation and accessibility tests.
4 - release	SVG export, decoder rounds, docs and change note.	Scannable exports, stable rendering, no unrelated file changes.

10. Public API boundary and acceptance

No prerequisite public data type or Scene API is required. BitGrid, Gf256, BitWriter and QrModuleKind belong in demo-local or private core modules. Promotion of BitGrid to the core library requires a second independent concrete use case and separate API/accessibility review; Phase 0 is explicitly not a public-library feature release.

Acceptance checklist

- Version 1-L byte-mode symbols conform to the QR specification; data/EC bytes, both format fields and all eight candidate matrices match independent fixtures.
- Final matrix is compared directly only when mask choices agree; otherwise the N3 convention exception and candidate correspondence are recorded.
- 59 input-independent snapshots can be revisited exactly, including 19 RS division iterations and 26 placed codewords.
- Snapshot 23 fills all fixed function modules via reserve() then fill_reserved(), while format cells stay unassigned until candidate-specific format placement at 50..57.
- Two bitsets plus optional cell roles enforce grid invariants; mask flips are restricted to Data/EC and each mask starts from one unmasked grid; bounded SVG paths show all cell states.
- User input changes recompute a Vec of exactly 59 snapshots and return to Unstarted; Input jump selects 0; external set_selection explicitly applies a snapshot without firing on_step.
- Interactive viewport is always 21 x 21; four light modules are added on every side only for export, which independent decoders successfully read.
- No public API changes, existing focus/zoom retained on routine updates, accessible stage summaries and keyboard controls supported.

References and repository checks

Normative: ISO/IEC 18004:2024, QR Code bar code symbology specification. Repository: https://github.com/ChrisWhealy/svg-dom-graph . Independent fixture tools: pinned Python segno and ZXing; optionally pyzbar.

API inspection on 29 October 2026: src/scene/selection_toolbar/mod.rs documents and calls Scene::set_selection, and states that external set_selection synchronises toolbar button availability. Scene::show_selection_toolbar resets its managed selection to None; its callback applies only to toolbar actions. Therefore external stage-jump and input-reset handlers must explicitly apply their own corresponding DemoSnapshot / Unstarted view.

This specification is a proposal only. Do not modify unrelated uncommitted files or historical Git commit text in changelog.md.